

Program 1

```
import pandas as pd
```

```
data = pd.DataFrame([
    ['Sunny','Warm','Normal','Strong','Warm','Same','Yes'],
    ['Sunny','Warm','High','Strong','Warm','Same','Yes'],
    ['Rainy','Cold','High','Strong','Warm','Change','No'],
    ['Sunny','Warm','High','Strong','Cool','Change','Yes']
], columns=['Sky','AirTemp','Humidity','Wind','Water','Forecast','EnjoySport'])
```

```
X = data.iloc[:, :-1].values
```

```
y = data.iloc[:, -1].values
```

```
n = X.shape[1]
```

```
# ---- Find-S Algorithm ----
```

```
hypothesis = ['0'] * n
```

```
for i in range(len(X)):
```

```
    if y[i] == 'Yes':
```

```
        for j in range(n):
```

```
            if hypothesis[j] == '0':
```

```
                hypothesis[j] = X[i][j]
```

```
            elif hypothesis[j] != X[i][j]:
```

```
                hypothesis[j] = '?'
```

```
print("Most Specific Hypothesis (Find-S):", hypothesis)
```

```
# ---- Candidate Elimination Algorithm ----
```

```
S = [hypothesis.copy()]
```

```
G = [['?'] * n]
```

```
for i in range(len(X)):
```

```
    if y[i] == 'Yes':
```

```
        G = [g for g in G if all(g[j] == '?' or g[j] == X[i][j] for j in range(n))]
```

```
    else:
```

```
        new_G = []
```

```
        for g in G:
```

```
            for j in range(n):
```

```
                if g[j] == '?':
```

```

        for value in set(X[:, j]):
            if value != X[i][j] and value == S[0][j]:
                new_hypothesis = g.copy()
                new_hypothesis[j] = value
                new_G.append(new_hypothesis)
    G = new_G

print("S (Specific Boundary):", S)
print("G (General Boundary):", G)

```

Program 2

```

import pandas as pd
from sklearn.datasets import load_iris
import wittgenstein as lw

iris = load_iris(as_frame=True)
X = iris.data
y = iris.target

# ---- RIPPER Algorithm ----
model = lw.RIPPER()
model.fit(X, y, pos_class=0) # class 0 = setosa
print("RIPPER Rules:")
print(model.ruleset_)

# ---- Simplified FOIL Algorithm ----
y_binary = (y == 0).astype(int) # 1 = setosa, 0 = not setosa

rules = []
for column in X.columns:
    for value in X[column].unique():
        rows_with_value = X[column] == value
        if y_binary[rows_with_value].all():
            rules.append(f"IF {column} = {value} THEN class = 1")

```

```
print("\nFOIL Rules (Setosa vs Not):")
for rule in rules[:6]:
    print(rule)
```

program 3

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# ---- Bagging ----
bag_model = BaggingClassifier(estimator=DecisionTreeClassifier(), n_estimators=50,
random_state=42)
bag_model.fit(X_train, y_train)
y_pred_bag = bag_model.predict(X_test)
print("Bagging Accuracy:", accuracy_score(y_test, y_pred_bag))
print("Bagging Confusion Matrix:\n", confusion_matrix(y_test, y_pred_bag))

# ---- Boosting (AdaBoost) ----
boost_model = AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1),
n_estimators=50, random_state=42)
boost_model.fit(X_train, y_train)
y_pred_boost = boost_model.predict(X_test)
print("\nBoosting Accuracy:", accuracy_score(y_test, y_pred_boost))
print("Boosting Confusion Matrix:\n", confusion_matrix(y_test, y_pred_boost))
```

Program 4

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

iris = load_iris(as_frame=True)
X = iris.data

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)
print("Cluster labels (first 5 samples):", clusters[:5])

plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=clusters, cmap='viridis', s=50)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            c='red', marker='X', s=200, label="Centroids")
plt.title("K-Means Clustering (Iris)")
plt.xlabel("Feature 1 (scaled)")
plt.ylabel("Feature 2 (scaled)")
plt.legend()
plt.show()
```